

Associate Flutter Developer - Practical Assessment

Product Catalogue Application

Objective

Build a small Flutter product catalogue application.

This assessment is designed to evaluate your:

- Flutter fundamentals
- UI and UX understanding
- State-management knowledge
- API integration ability
- Navigation knowledge
- Code organisation
- Git workflow
- Problem-solving approach

The application does not need to be highly advanced. We are more interested in how you approach the work, organise the project, and explain your decisions.

Core Requirements

1. Product List Screen

Create a screen that displays a list or grid of products.

Each product should display:

- Product image
- Product name
- Price
- Category
- Favourite indicator or button.

2. Product Details Screen

When the user selects a product, The details screen should display.

- Larger product image
- Product name
- Price
- Category
- Full product description
- Favourite button

3. Search

Add a search field to the product list screen.

The search must:

- Search using the product name
- Support substring matching
- Update the visible product list as the user types

4. Favourites

Allow users to add and remove products from their favourites.

The favourite status should update correctly on both:

- Product list screen
- Product details screen

Optional Bonus

Persist the favourite products after the application is closed and reopened.

5. State Management

Use appropriate ways for state-management for this assessment.

6. Light and Dark Themes (Optional)

Implement:

- Light theme
- Dark theme
- A way for the user to switch between them

The theme should affect the overall application instead of changing only one or two widget colours.

7. Product Data

Load product data mocked manner is accepted. Better approaches are accepted.

Do not add unnecessary backend functionality. If you want to add a small local backedn. Try to be simple an add only to demonstrate yourself. Do not over architect

8. Loading and Error Handling

When loading products, display an appropriate loading state.

If the product data cannot be loaded, display:

- A clear error message
- A retry option

The application should not remain on an infinite loading indicator or crash when the request fails.

An empty state should also be displayed when

- No products are available
- The search returns no matching products

Version control and commit history will give you some bonus marks

We are not checking whether your commit history exactly matches the examples. We want to see a genuine and understandable development process.

Do not rewrite or remove the development history before submission. Just take time and do the work consistently.

UI and UX Expectations

The design does not need to exactly match an existing application.

However, the application should demonstrate:

- Consistent spacing
- Readable text
- Appropriate image sizing
- Clear buttons and icons
- Proper handling of long product names

- Proper handling of long descriptions
- Consistent light and dark themes
- Useful loading, empty, and error states

We are interested in whether you improve the default Flutter project and create a reasonably polished user experience.

Simply placing all content inside default widgets without considering layout, theme, spacing, or usability may negatively affect the evaluation.

README Requirements

Add a README.md file containing

Project Overview

Briefly explain what the application does.

Setup Instructions

Explain how to:

- Install dependencies
- Run the project
- Build an APK

Include the relevant commands,

Architecture

Briefly explain:

- Folder structure
- State-management approach
- API integration approach

Assumptions

Mention any assumptions made while completing the assignment.

Challenges

Mention any difficulties encountered and how they were handled.

Improvements

Mention any Improvement can be added to the project in short.

Submission Requirements

Submit:

- Git repository link
- APK file with a Demo video(mandatory)
- Completed README
- Screenshots or a short screen recording of the application. Better if you add them to readme as well.

Make sure the repository includes the complete Git commit history.

Expected Scope

Keep the project simple.

Do not spend time implementing unnecessary features. Completing the core requirements properly is more valuable than adding many incomplete features.